

Gdańsk University of Technology
Faculty of Electronics, Telecommunications and Informatics
Department of Multimedia Systems

Technical Documentation Mobile Application (iOS)

v1.0

iOS Application - Full Technical Overview (v1.0)

Project Subject: Glaucoma detection using intelligent analysis of fundus images

Team Members: Martyna Borkowska (Lead), Karolina Glaza, Mateusz Dobry, Agnieszka Pawłowska, Amila Amarasekara, Antoni Naczke

Supervisor: Prof. Andrzej Czyżewski, Ph.D., D.Sc., Eng.

Author: Karolina Glaza

Contents

1	Introduction	2
2	Scope of Responsibility	2
3	iOS Application Architecture	2
3.1	MVVM Pattern	2
3.2	Navigation - the <code>AppScreen</code> enum	2
4	Backend Communication	3
4.1	NDJSON Streaming	3
4.2	Data Model	4
4.3	Server-side Image Processing	4
5	Application Views	5
6	Visual Consistency	5
6.1	Colour Palette (Dark UI)	5
6.2	Reusable Components	6
7	Testing	6
7.1	Swift Unit Tests (XCTest)	6
7.2	Backend Tests (pytest)	7
8	Current State and Plan	7
9	Summary - Deliverables	8

1 Introduction

The iOS application is the user-facing component of the glaucoma screening system. A physician captures or selects a fundus photograph, the application sends it to the backend for AI analysis (YOLO region-of-interest detection → UNet++ segmentation → Cup-to-Disc Ratio), and then presents the diagnosis. The diagnostic threshold used by the system is $CDR \geq 0.65$ (suspected glaucoma) and $CDR \geq 0.70$ (high risk).

This document is a full technical overview of **version 1.0** of the application: its architecture, backend communication, views, visual design system, and test coverage.

2 Scope of Responsibility

Thesis topic: *Software for automatic segmentation of fundus structures for the early detection of glaucoma.*

- iOS mobile application in Swift (Swift Playgrounds).
- API communication layer - NDJSON streaming.
- FastAPI backend on Microsoft Azure.
- Swift unit tests (XCTest).
- Backend tests (pytest).
- Repository, project management, CI/CD.

3 iOS Application Architecture

3.1 MVVM Pattern

The application follows the Model-View-ViewModel (MVVM) pattern. A single `GlaucomaViewModel` (annotated `@MainActor`) owns the application state and coordinates communication with the backend service. `@MainActor` guarantees that UI updates run on the main thread;

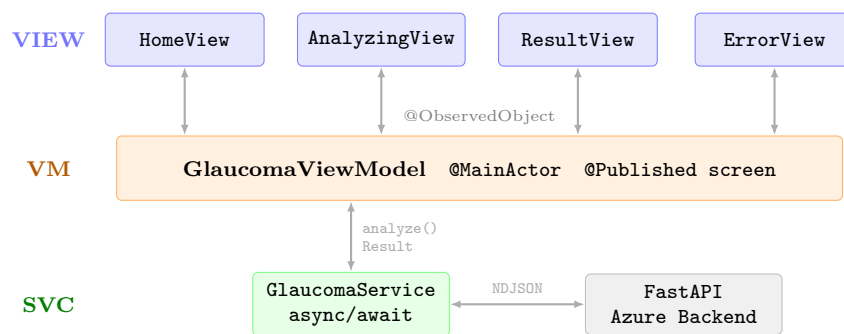


Figure 1: MVVM architecture of the iOS application.

`async/await` moves heavy AI tasks into the background; navigation is modelled as an `enum AppScreen`.

3.2 Navigation - the AppScreen enum

Navigation is modelled as a Swift `enum AppScreen` with four cases: `.home`, `.analyzing`, `.result(GlaucomaResult)`, and `.error(String)`. State transitions are driven exclu-

sively by the ViewModel.

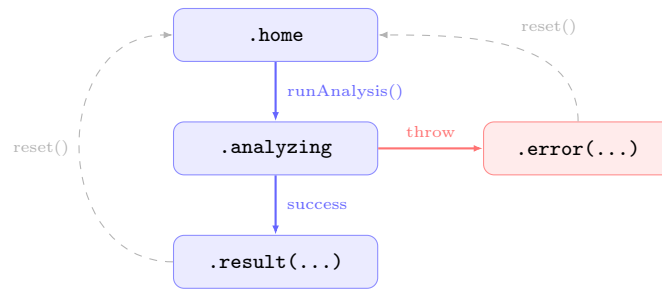


Figure 2: Navigation state machine based on enum AppScreen.

Listing 1: ContentView.swift - state-driven view switching

```

switch vm.screen {
case .home:
    HomeView(vm: vm)
case .analyzing:
    AnalyzingView(vm: vm)
case .result(let res):
    ResultView(vm: vm, result: res)
case .error(let msg):
    ErrorView(message: msg, vm: vm)
}

```

There is no NavigationStack: the navigation state is fully controlled by the ViewModel, and transitions are animated with `.animation(.easeInOut)`.

4 Backend Communication

4.1 NDJSON Streaming

The analysis result is streamed from the backend as Newline-Delimited JSON (NDJSON), with one event per pipeline step:

1. Image received.
2. Loading AI models (cold start only).
3. AI inference.
4. Mask processing.
5. Success (final result).

Listing 2: GlaucomaService.swift - consuming the NDJSON stream

```

let (asyncBytes, _) = try await URLSession.shared.bytes(for: request)

for try await line in asyncBytes.lines {
    let progress = try JSONDecoder().decode(StreamProgress.self, from: line)
    if let step = progress.step {
        await MainActor.run { onStep(step) }
    }
    if progress.status == "success" {
        finalResult = progress.data
    }
}

```

```
}

```

Why streaming? The AI analysis can take 5-15 seconds. NDJSON updates the UI continuously, preventing the “frozen screen” effect.

4.2 Data Model

The response is decoded into typed Swift structures. `CodingKeys` map the backend’s `snake_case` fields to idiomatic camelCase Swift properties; the entire stream is decoded with a single `JSONDecoder`.

Listing 3: GlaucomaService.swift - data model

```
struct GlaucomaResult: Codable {
    let hasGlaucoma: Bool
    let confidence: Double
    let cupToDiscRatio: Double
    let imageBase64: String

    enum CodingKeys: String, CodingKey {
        case hasGlaucoma = "has_glaucoma"
        case confidence
        case cupToDiscRatio = "cup_to_disc_ratio"
        case imageBase64 = "image_base64"
    }
}

struct StreamProgress: Decodable {
    let status: String
    let step: Int?
    let message: String?
    let data: GlaucomaResult?
}
```

Listing 4: API contract - final response (JSON)

```
{
  "status": "success",
  "step": 5,
  "data": {
    "has_glaucoma": true,
    "confidence": 0.95,
    "cup_to_disc_ratio": 0.71,
    "image_base64": "<JPEG base64>"
  }
}
```

4.3 Server-side Image Processing

For completeness, the backend post-processes the pipeline output before returning it:

- **CDR > 0.65** → `has_glaucoma = True`.
- **Mask overlay** - disc pixels tinted green, cup pixels red (50/50 blend with OpenCV).
- The image is **Base64-encoded** and returned in the `image_base64` field.

- `run_in_threadpool` - AI inference runs in a separate thread so it does not block the FastAPI event loop.

Listing 5: Backend - thresholding, mask overlay, threaded inference

```
# Diagnostic threshold
cup_to_disc_ratio = round(float(cdr_val), 2)
is_glaucoma = bool(cup_to_disc_ratio > 0.65)

# Blend masks onto the pixels (OpenCV)
roi[disc_resized > 0.5] = (roi[disc_resized > 0.5] * 0.5
                          + np.array([0, 255, 0]) * 0.5)    # green
roi[cup_resized > 0.5] = (roi[cup_resized > 0.5] * 0.5
                          + np.array([255, 0, 0]) * 0.5)    # red

# AI inference in a separate thread (streaming)
result = await run_in_threadpool(glaucoma_pipeline.run, tmp_path)
```

5 Application Views

The application exposes four screens, each driven by the `AppScreen` state.

HomeView

- Image source selection: `PhotosPicker` (gallery) or `UIImagePickerController` (camera).
- Preview of the selected photo with the option to change it.
- The **ANALYZE** button becomes active only after an image is selected.

AnalyzingView

- Animation - a rotating arc and a pulsing eye icon.
- Four UI steps driven by `vm.analysisStep`.

ResultView

- Diagnostic banner: *GLAUCOMA DETECTED / NO GLAUCOMA SIGNS*.
- Processed image with the segmentation overlay (disc/cup).
- Three `MetricCards`: CDR, confidence, risk level.
- `CDRScaleView` - a 0.0-1.0 scale with a gradient.

ErrorView

- Animated error icon (spring animation).
- *Try again* - re-triggers `runAnalysis()`.
- *Back* - returns to `HomeView`.

6 Visual Consistency

6.1 Colour Palette (Dark UI)

All colours are defined as `static let` in an extension `Color` - a single place to re-theme the app without searching for literals in the code.

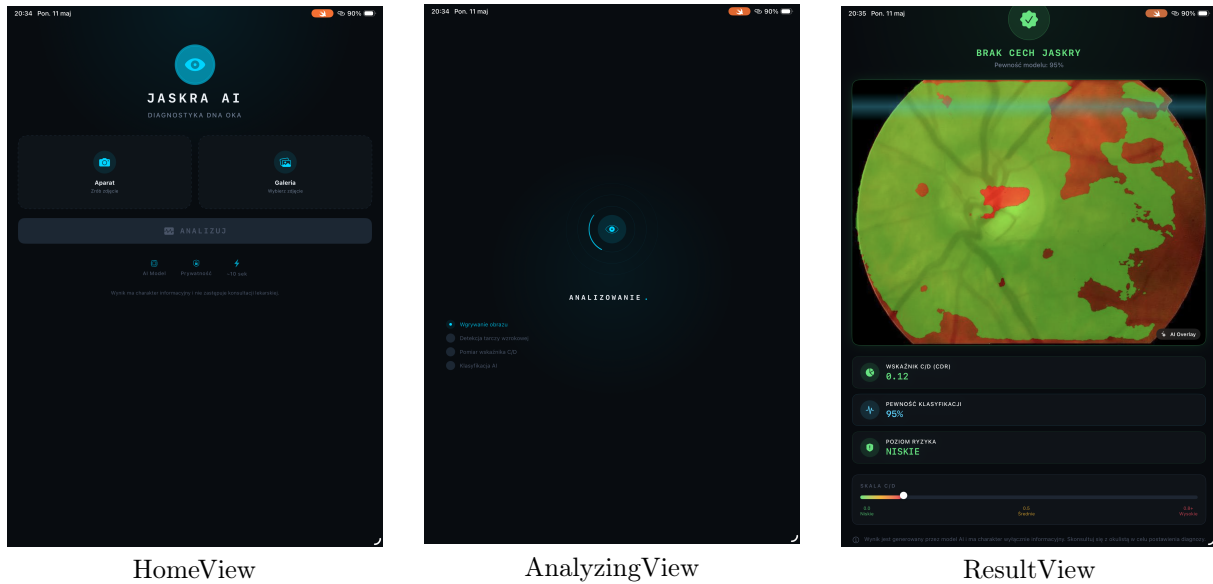


Figure 3: Application screens (v1.0).

Token	Hex	Usage
appBackground	#080C10	global application background
cardBackground	#0E1419	cards and panels
accentCyan	#00D4FF	accent, active UI elements
positiveGreen	#00E676	no glaucoma, low CDR
warningAmber	#FFB300	moderate CDR (0.4-0.6)
dangerRed	#FF3D5A	glaucoma detected, $CDR \geq 0.6$
textPrimary	#EDF2F7	headings and primary text

Table 1: Application colour tokens.

6.2 Reusable Components

- **MetricCard** - icon + label + value with a coloured accent.
- **GlowBorder** - a `ViewModifier` drawing a glowing `RoundedRectangle` border.
- **PulsingCircle** - an animated circle (pulse scale + opacity).
- **ScanLineEffect** - a moving gradient (AI overlay).
- **Color(hex:)** - initializer for `#RGB`, `#RRGGBB`, `#AARRGGBB`.
- **CDRScaleView** - a gradient track with a thumb proportional to the CDR; colours change dynamically (green/amber/red).
- **DiagnosisBanner** - the **GLAUCOMA DETECTED** / **NO SIGNS** banner with an animated entrance.

7 Testing

7.1 Swift Unit Tests (XCTest)

`GlaucomaViewModelTests.swift` - 11 test cases, executed under `@MainActor`.

Test	Input (CDR)	Expected result
test_risk_text_negative	0.0, 0.3	"NISKIE" (low)
test_risk_text_boundary_low	0.4	"UMIARKOWANE" (moderate)
test_risk_text_moderate	0.5	"UMIARKOWANE" (moderate)
test_risk_text_boundary_high	0.6	"WYSOKIE" (high)
test_risk_text_positive	0.7, 1.0	"WYSOKIE" (high)
test_risk_color_negative	0.0, 0.3	.positiveGreen
test_risk_color_boundary_low	0.4	.warningAmber
test_risk_color_moderate	0.5	.warningAmber
test_risk_color_boundary_high	0.6	.dangerRed
test_initial_state	-	nil, step=0, picker=false
test_reset_clears_state	-	nil after reset()

Table 2: iOS unit tests. Boundary values 0.4 and 0.6 are tested explicitly as the UI decision thresholds.

7.2 Backend Tests (pytest)

Unit tests (test_api.py) - mock `GlaucomaPipeline.run()` with the FastAPI Test-Client.

Test	Verifies
test_success_positive	CDR=0.70 → has_glaucoma=True
test_success_negative	CDR=0.40 → has_glaucoma=False
test_pipeline_none	no detection → zeros (0.0)
test_cdr_boundary	CDR=0.65 → False (strict > threshold)
test_cdr_edge_values	CDR=0.0 and 1.0 ∈ [0, 1]

Validation tests (test_validation.py)

Test	Expected
missing file	HTTP 422 (Unprocessable Entity)
text file	validation error ($\neq$ 200)
empty JPEG	decoding error ($\neq$ 200)
corrupted JPEG	processing error ($\neq$ 200)

Streaming tests (test_streaming.py)

Integration tests (test_integration.py) - run against a live server (`pytest -m integration`): full JSON structure (HTTP 200), value ranges (CDR, confidence ∈ [0, 1]), Base64 decodability of the result image, and HTTP 422 on missing input.

8 Current State and Plan

In progress:

- Swift/iOS tests - XCTest (ViewModel, reset, threshold boundaries).

Test	Verifies
<code>test_stream_200</code>	HTTP 200 OK
<code>test_stream_steps</code>	≥ 1 event with a <code>step</code> field
<code>test_final_success</code>	last event <code>status="success"</code>
<code>test_steps_ordered</code>	steps ascending ($1 \rightarrow 5$)
<code>test_five_steps</code>	last step = 5

- AI pipeline, YOLO and UNet++ tests (in collaboration).
- Camera work.

To do:

- Codecov - integration after the test branches are merged.
- Clinical validation (Dr. Szafik's data).
- Integration with clinical hardware.
- Image metadata cleaning before the POST request in Swift.

9 Summary - Deliverables

1. **iOS application** ready for use by a physician (Swift Playgrounds, iOS 18.1+, iPhone + iPad).
2. **NDJSON streaming** - analysis progress without a frozen screen.
3. **Visual consistency** - a dark medical style with a centralised colour system and reusable components.
4. **Test coverage:** XCTest (Swift) + pytest (backend).

Together with the AI model (currently $\approx 90\%$ accuracy), version 1.0 delivers an end-to-end, doctor-usable glaucoma pre-screening application.